

二进制常见trick

- 快速判断奇偶： $n \& 1$
 - 乘除 $2^n \gg 1, n \ll 1$
 - $x \gg k$ 是指x右移k格就是把最右边顶掉就是除以二
 - 正数是向下取整负数是向下（比如 $-5 \gg 1 = -3$ ）
- 关于2的次方：必须注意的两个“坑”坑一：优先级问题位运算的优先级低于加减法！错误： $1 \ll k - 1$ (会被解析为 $1 \ll (k - 1)$)错误： $a + 1 \ll k$ (会被解析为 $(a + 1) \ll k$)正确：总是加括号！ $(1 \ll k)$ 坑二：不要用 $\text{pow}(2, k)$ $\text{pow}()$ 函数是浮点运算，返回值是 `double`。缺点：慢：比位运算慢得多。精度问题：当 k 很大时，转回 `int` 或 `long long` 可能会出现精度丢失（比如算出 31.99999 转成 31）。结论：整数运算永远只用 $\ll$ 。
- 是不是2的整数次幂
 - 作用： $\log_2 n$;
 - $n \& (n - 1)$
 - lowbit:用于树状数组update 和 query 操作balabala反正我不会
 - 作用：得到 n 的二进制表示中，最低位的 1 所代表的数值
 - $n \& (-n)$
 - 位掩码(用一个整数来代表一个集合)
 - 检查第k位是不是1: $\text{if } (n \& (1 \ll k))$
 - 将第k位变成1 $n = n \mid (1 \ll k)$
 - 反转第k位 $n = n \wedge (1 \ll k)$
 - 把第k位变成0 $n = n \& \sim(1 \ll k)$;
 - 获取 a 的第 b 位 (0或1) $(a \gg b) \& 1$
 - 快速获取第k位 $(hsh \gg k) \& 1$

教学局

&

- 作用：只保留两个都是1的位置
- 特征 $a_i \& a_j \neq 0$ -> 二进制的每一位 (0~30)

^

- 规律
 - $x \wedge x = 0$ (自我抵消)
 - $x \wedge 0 = x$ (保持不变)
 - $A \wedge B \wedge C = C \wedge A \wedge B$ (满足交换律和结合律)
- 作用：解决各种“成对抵消”问题:区间异或和

|

- 或，有1就是1

~a

- 按位取反 $\sim x = -x - 1$

常见处理方法

拆位计算贡献

- 对于一个数组来说可能 $O(n^2)$ 复杂度太高了，我们需要进行拆位操作(或是用拆位贡献的思想来思考问题)
- 或者题目直接要求你计算一堆数字异或起来/和起来/或起来
- 把加法的进位完全忽略掉来做拆位最后再乘上每一位对应的 $1 \ll i$ 是非常常见的做法
- 思考方式：
 - 对于每一位，他能对答案产生多少贡献？
 - 贡献的是怎么算的？
 - 求的是谁？怎么组合？

异或序列示例1

- 题目

给出序列 $A_1, A_2, \dots, A_N$ ，求

$$\sum_{1 \leq i \leq j \leq N} A_i \oplus A_{i+1} \oplus \dots \oplus A_j$$

的值。其中， $\oplus$ 表示按位异或。

- 对于 60 的数据， $1 \leq N \leq 10^3$ ；

- 对于 100 的数据, $1 \leq N \leq 10^5$, $0 \leq A_i \leq 10^9$ 。

• 思考方式

- 这里求着连续子段异或和 -> xor前缀和
- 这里对每个连续子段异或和的和 -> 用xor前缀和拆位
- 拆位常见算贡献在那一位上面的cnt0*cnt1(那一位上的排列组合的种类数)(XOR)
- 这道题特殊就特殊在他要用前缀异或和去算拆位, 因为每一段前缀异或和都要算 $2^{(n-1)}$ 遍 (如果别人需要取中间的某一段就需要那一段贡献右端点的和贡献左端点的进行异或)

• AC代码

```
void solve()
{
    int n;
    cin >> n;
    vi nums(n);

    vi cnm(n);
    rep(i, 0, n - 1)
    {
        cin >> nums[i];
        if (!i)
            cnm[0] = nums[0];
        else
            cnm[i] = nums[i] ^ cnm[i - 1];
    }
    vector<vi> bits(32, vi(2));
    for (int hsh : cnm)
    {
        for (int i = 0; i < 31; i++)
        {
            if ((hsh >> i) & 1)
                bits[i][1]++;
            else
                bits[i][0]++;
        }
    }

    ll ans = 0;
    for (int i = 0; i < 31; i++)
    {
        ans += (ll)bits[i][1] * (bits[i][0] + 1) * (1ll << i); // +1是空集
    }
    cout << ans;
}
```

C. Divan and bitwise operations 示例2

- 模型: 给一个数列, 求他们子集异或和的和
- 思考
 - 跟上题完全一样, 现在我们要想的是, 对于某个数字的某一位他们的01是怎么参与贡献的?
 - 这里涉及到一点数学上的小处理: 如果那个位置上的01串里面有1就有 $2^{(k-1)}$ 个组合方式。
- 题目
 - 给出一个数列若干个区间 l_r 的区间异或和, 求这个数列的非空子集的异或和

Once Divan analyzed a sequence $a_1, a_2, \dots, a_n$ consisting of n non-negative integers as follows. He considered each non-empty subsequence of the sequence a , computed the **bitwise XOR** of its elements and added up all the XORs, obtaining the coziness of the sequence a . A sequence c is a subsequence of a sequence d if c can be obtained from d by deletion of several (possibly, zero or all) elements. For example, $[1, 2, 3, 4]$, $[2, 4]$, and $[2]$ are subsequences of $[1, 2, 3, 4]$, but $[4, 3]$ and $[0]$ are not. Divan was very proud of his analysis, but now he lost the sequence a , and also the coziness value! However, Divan remembers the value of **bitwise OR** on m contiguous subsegments of the sequence a . It turns out that each element of the original sequence is contained in **at least one** of these m segments. Divan asks you to help find the coziness of the sequence a using the information he remembers. If several coziness values are possible, print any. As the result can be very large, print the value modulo $10^9 + 7$.

有一次, Divan分析了一个由

$$n$$

个非负整数组成的序列

$$a_1, a_2, \dots, a_n$$

, 如下所示。他考虑序列

$$a$$

\$ 的每个非空子序列，计算其元素的位异或并将所有异或相加，得到序列	
	$\$a$
的舒适度。序列	
	c
是序列	
	d
的子序列，如果	
	c
可以通过删除	
	d
中的几个（可能为零或全部）元素来获得。例如，	
	$[1, 2, 3, 4]$
,	
	$[2, 4]$
和	
	$[2]$
是	
	$[1, 2, 3, 4]$
的子序列，但	
	$[4, 3]$
和	
	$[0]$
不是。Divan对他的分析非常自豪，但是现在他失去了序列	
	a
, 也失去了舒适度! 然而, Divan记住了序列	
	a
的连续子段	
	m
\$ 上的位或的值。结果表明，原始序列的每个元素至少包含在这些	
	$\$m$
片段中的一个 * * 中。Divan要求您使用他记住的信息帮助找到序列	
	a
的舒适性。如果有多个舒适值，打印任意一个。由于结果可能非常大，因此打印值modulo	
	$10^9 + 7$
\$ 。	

- 思路
 - 这里直接就涉及到一个ai出来的二级结论，由于我已经先入为主入脑入心入魂所以我直接用了，但是我会把推导过程写出来
 - 二级结论: $total_or * 2^{(n-1)}$
- 思路：对于这个我们推导出：如果那个位置上的01串里面有1就有 $2^{(k-1)}$ 个组合方式。。那么 or运算可以帮你找到哪些位置上有至少一个1

ABC某神秘题目

- 问题陈述
 - 我们有 N 个整数。 i \ 第一个整数是 A_i 。
 - 找到 $\sum_{i=1}^{N-1} \sum_{j=i+1}^N (A_i \text{ XOR } A_j)$, 模数 $(10^9 + 7)$ 。
- 数据范围### Constraints

- $2 \leq N \leq 3 \times 10^5$
 - $0 \leq A_i < 2^{60}$
- **注意事项：** 这种数据范围这么大的请开qpow! !，记得每次取模

```
// 计算 (a ^ b) % p
ll qpow(int a, int b, int p) {
    ll res = 1;
    a %= p; // 防止一开始 a 就大于 p 的情况
    while (b > 0) {
        // 如果 b 的当前末位是 1, 则乘上当前的 a
        if (b & 1) {
            res = (res * a) % p;
        }
        // a 自乘, 准备计算下一位
        a = (a * a) % p;
        // b 右移一位 (相当于除以 2)
        b >>= 1;
    }
    return res;
}

void solve()
{
    int n;
    cin >> n;
    vi a(n);
    rep(i, 0, n - 1)
    {
        cin >> a[i];
    }
    vi dp(64);
    rep(k, 0, 63)
        rep(i, 0, n - 1)
        {
            dp[k] += ((a[i] >> k) & 1);
        }

    ll ans = 0;

    rep(k, 0, 63)
    {
        ans=ans%MOD+(1ll)(((dp[k]%MOD)*((n-dp[k])%MOD))%MOD*qpow(2,k,MOD))%MOD;
    }
    cout<<ans;
}
```

C_Renako_Amaori_and_XOR_Game 示例3

- **题目：** 给出两个数组a和b，两个人博弈，先手操纵奇数列，后手操作偶数列。操作时可以交换两个数组对应同列（同下标）的两个项。胜利条件:先手拥有数组a后手拥有数组b，数组里面所有项异或和最大获胜。求谁有必胜策略？ 如果平局输出Tie

这是这个问题的难点。简单版本和硬版本的唯一区别是，在硬版本中，

$$a_i, b_i \leq 10^6$$

. * * Renako被困在岩石和坚硬的地方之间...当然，我指的是Ajisai和Mai！他们俩都想和她一起出去玩，而她就是拿不定主意！所以Ajisai和Mai决定玩XOR游戏

$$n$$

(

$$0 \leq a_i, b_i \leq 10^6$$

) 的数组

$$a$$

和

$$b$$

。他们将玩一个回合的游戏，Ajisai在奇数回合移动，Mai在偶数回合移动。在

$$i$$

第1回合，要移动的玩家可以选择交换

$$a_i$$

和

$$b_i$$

，或者通过。注意，如果发生交换，被交换的索引必须与轮数匹配。例如，在第一回合，味斋可以选择交换

$$a_1$$

和

$$b_1$$

，或者通过。在第二回合，*Mai*可以选择交换

$$a_2$$

和

$$b_2$$

，或者通过。

$$n$$

轮继续这样。因此，只有*Ajisai*可以交换奇指标，而只有*Mai*可以交换偶指标。游戏结束时，*Ajisai*得分为

$$a_1 \oplus a_2 \oplus \cdots \oplus a_n$$

，*Mai*得分为

$$b_1 \oplus b_2 \oplus \cdots \oplus b_n$$

*

。得分较高的玩家获胜。如果双方得分相同，比赛以平局结束。用最优玩法确定游戏的结果。更正式地说，如果存在一种策略，让一个玩家无论对手的选择如何都能获胜，

*

$$\oplus$$

\$表示[按位异或运算](#)

- 分析:这道题目有两个难点:博弈和XOR。我们先一个一个来解析
 - XOR位运算: 博弈: 学了bash博弈都知道我们要找到的是决胜局的条件: 谁能操控决胜局/先手能不能一直让后手必败
 - 这里是要找对于结果 (数组异或和) 大于对面这个条件的贡献。有一个特殊情况是所有数字如果xor在一起是0那么前一步A^A=0两个数组异或和必然相等。这是平局情况
 - 对于获胜情况而言: 谁能找到最后一个可以获得控制所有数字异或和最高位数字谁就能赢
- 因为在最后一步的最高位(highbit)一定是由前面一步的1^0获得。要不然不存在
- 此时问题变得很明了:我们怎么知道最后一个可以控制所有数字异或和的最高位呢?
 - 最后一个控制所有数字异或和的最高位必然有:他们自己异或起来最高位是1，因为他们需要在这个位置上同时有0和1两种情况出现他才有能力改变他自己 (如果在这一位上面cnt1累计是奇数需要0来填充，累计是偶数需要1来填充)

```
void solve()
{
    int n;

    cin >> n;
    vi a(n+1);
    vi b(n+1);
    rep(i, 1, n )
    {
        cin >> a[i];
    }
    rep(i, 1, n )
    {
        cin >> b[i];
    }
    int sum=0;
    int c2=0;
    rep(i, 1, n )
    {
        sum^=b[i];
        sum^=a[i];
    }
    int gao=0;
```

```

for(int i=31;i>=0;i--)
{
    if((sum>>i)&1)
    {
        gao=i;
        break;
    }
}

for (int i = n; i >= 1; --i) {
    if (((a[i] ^ b[i]) >> gao) & 1) {
        c2 = i;
        break;
    }
}
// cerr<<gao<<' ';
if(!sum){cout<<"Tie"<<'\\n';}
else if(c2%2){cout<<"Ajisai"<<'\\n';}
else if(c2%2==0){cout<<"Mai"<<'\\n';}
}

```

P6599 「EZEC-2」异或

• 题目

有 T 组询问，每次给定两个正整数 n, l ，你需要构造一个长度为 l 的正整数序列 a （编号从 1 至 l ），且满足 $\forall i \in [1, l]$ ，都有 $a_i \in [1, n]$ 。求

$$\sum_{i=1}^l \sum_{j=1}^{i-1} a_i \oplus a_j$$

的最大值。为了避免答案过大，对于每组询问，只需要输出这个最大值对 $10^9 + 7$ 取模的结果。

【数据规模与约定】 | 测试点编号 | $T \leq$ | $n \leq$ | $l \leq$ | :-----: | :-----: | :-----: | :-----: | | 1 ~ 5 | 1 | 10 | 5 | | 6 | 5×10^5 | 10^{12} | 2 | | 7 | 5×10^5 | 10^{12} | 3 | | 8 | 5×10^5 | 10^{12} | 10^5 |

对于 100 的数据，满足 $1 \leq T \leq 5 \times 10^5$ ， $1 \leq n \leq 10^{12}$ ， $2 \leq l \leq 10^5$ 。

• 解析

- 题目非常简单，只需要一点点贪心就能做
- 注意最后的处理（恶心的我半小时） $ans += (11) (((a \% MOD * b \% MOD) \% MOD) * ((11) (111 < i) \% MOD)) \% MOD$ ；其中
 1. $(1 < i)$ 默认是 `int` 类型的，请对它里面的 1 进行强制类型转换
 2. 或者用等差数列求和的巧妙小技巧 $ans += (11) (((a \% MOD * b \% MOD) * (qpow(2, high_bit + 1, MOD) - 1 + MOD)) \% MOD)$ ；注意这里要 +1，-1 具体为等差数列求和公式
- $== highbit$ 算位置一直都是从 0 开始不需要质疑 ==
- // 特判一下 $n=1$ ，这在二进制题目里面很常见

```

void solve()
{
    int n, l;
    cin >> n >> l;
    if (n == 0 || n == 1) { // 特判一下 n=1，这在二进制题目里面很常见
        cout << 0 << '\\n';
        return;
    }
    ll a = (1 >> 1);
    ll b = 1 - a;
    int high_bit = 0;
    for (int i = 63; i >= 0; i--)
        // rep(i, 0, 63)
        {
            if ((n >> i) & 1) { high_bit = i;
                break; }
        }
    ll ans = 0;
    for (int i = 0; i <= high_bit; i++)
        ans += (11) (((a \% MOD * b \% MOD) \% MOD) * ((11) (111 < i) \% MOD)) \% MOD;
    cout << ans \% MOD << '\\n';
}

```

例题

异或运算

子段异或和

- 题号: D
- 链接: [题目链接](#)
- 算法类型: 异或, 前缀和
- AC 代码:

```
void solve()
{
    int n;
    cin >> n;
    vi nums(n + 1);
    nums[0] = 0;
    vi cf(n + 1);
    map<int, int> bg;
    int ans = 0;
    bg[0]++;
    for (int i = 1; i <= n; i++)
    {
        cin >> nums[i];
        cf[i] = cf[i - 1] ^ nums[i];
        int key = cf[i] ^ 0; // a^b=0 -> a^0=b
        if (bg.find(key) != bg.end())
        {
            ans += bg[key];
        }
        bg[cf[i]]++;
    }
    cout << ans;
}
```

- 异或前缀和思路:

- 异或的“逆运算”就是它自己。
- 所以通过求异或前缀和来实现对 $[l, r]$ 区间的异或和的 $O(1)$ 查找——>就等于 $cf[r] \oplus cf[l-1]$ 。
- 异或的一些小妙招:
 - 寻找出现次数偶数/奇数次数的数列里面出现奇数/偶数次数的数
 - N堆石子, 每次可以取任意多个至少一个。异或和 $\neq 0$ 先手必败

- 改进技巧: 哈希表:

- $O(N)$ 优化 (使用哈希表): 我们用一个哈希表 countMap 存储历史上的前缀和 $cf[j]$ 出现过的次数 (复杂度 $O(\log N)$)
- 能 $O(1)$ 查询的字段和都可以采用

“前缀和+哈希表”是处理“子数组/子段”的小妙招

线性基板题